SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO2: AT1_INDUSTRY PROBLEM TASK

concepts to model decision-making problems. (BL2,BL3)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

STUDENT.NAME:-M.GUNA SHEKAR

REG.NO:-192472345

CASE STUDY 1:

Reinforcement Learning for Delivery Route Optimization in Logistics

Aim

To develop a Reinforcement Learning-based intelligent delivery routing system that minimizes travel time, fuel consumption, and delivery delays while maximizing delivery efficiency.

Objective

- Reduce delivery time.
- Minimize fuel consumption.
- Find the shortest and safest delivery route.
- Improve customer satisfaction.
- Adapt to real-time traffic conditions.

Problem Statement

Traditional routing systems use fixed shortest-path algorithms that cannot effectively adapt to changing traffic conditions or unexpected roadblocks. Reinforcement Learning enables delivery vehicles to continuously learn and select optimal routes based on real-time environmental conditions.

Software Requirements

- Python 3.10+
- VS Code
- NumPy
- Matplotlib

Process

1. Initialize the delivery environment.
2. Load delivery locations.
3. Observe the vehicle's current state.
4. Select the next route using the RL policy.
5. Move the vehicle.
6. Calculate reward.
7. Update the Q-table.
8. Repeat until all deliveries are completed.
9. Store the learned policy.

State Space

- Vehicle location
- Traffic level

- Fuel level
- Remaining deliveries

Action Space

- Move Left
- Move Right
- Move Forward
- Take Alternate Route

Reward Function

- +100 → Successful delivery
- +20 → Shortest path
- -10 → Traffic delay
- -20 → Fuel wastage
- -50 → Wrong route

Algorithm (Q-Learning)

1. Initialize Q-table.
2. Observe the current state.
3. Choose an action using ε-greedy.
4. Execute the action.
5. Receive reward.
6. Observe the next state.
7. Update the Q-value.
8. Repeat until destination is reached.
9. Save the optimal policy.

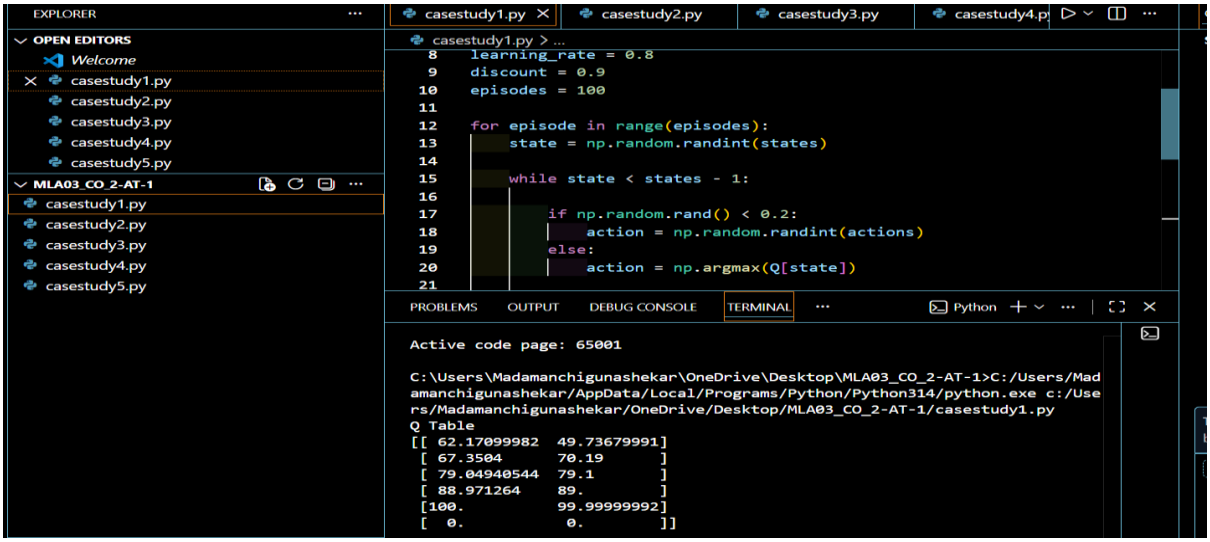
Expected Output

- Optimal delivery route
- Reduced delivery time
- Lower fuel consumption
- Improved logistics efficiency

Applications

- Amazon
- Flipkart
- DHL

- FedEx
- Swiggy
- Zomato
- CODE:-



Advantages

- Learns automatically.
- Adapts to traffic.
- Saves fuel.
- Faster deliveries.
- Reduces operational cost.

Conclusion

The RL-based delivery routing system continuously learns from traffic and delivery experiences, providing efficient and cost-effective logistics management

CASE STUDY 2: Reinforcement Learning for Fraud Detection

Title

Fraud Detection System Using Reinforcement Learning

Aim

To build an intelligent fraud detection system using Reinforcement Learning that accurately identifies fraudulent transactions while minimizing false positives.

Objective

- Detect fraud quickly.
- Reduce financial losses.
- Improve transaction security.
- Minimize false alarms.
- Learn new fraud patterns automatically.

Problem Statement

Conventional fraud detection systems rely on predefined rules, making them ineffective against evolving fraud techniques. Reinforcement Learning continuously adapts by learning from transaction outcomes.

Software Requirements

- Python
- NumPy
- Pandas
- VS Code

Process

1. Collect transaction data.
2. Observe transaction features.
3. Select an action.
4. Verify transaction.
5. Receive reward.
6. Update policy.
7. Continue learning.

State Space

- Transaction amount
- Customer history
- Device information

- Location
- Transaction frequency

Action Space

- Approve
- Reject
- Request OTP

Reward Function

- +100 → Fraud detected
- +50 → Correct approval
- -100 → Fraud missed
- -30 → False rejection

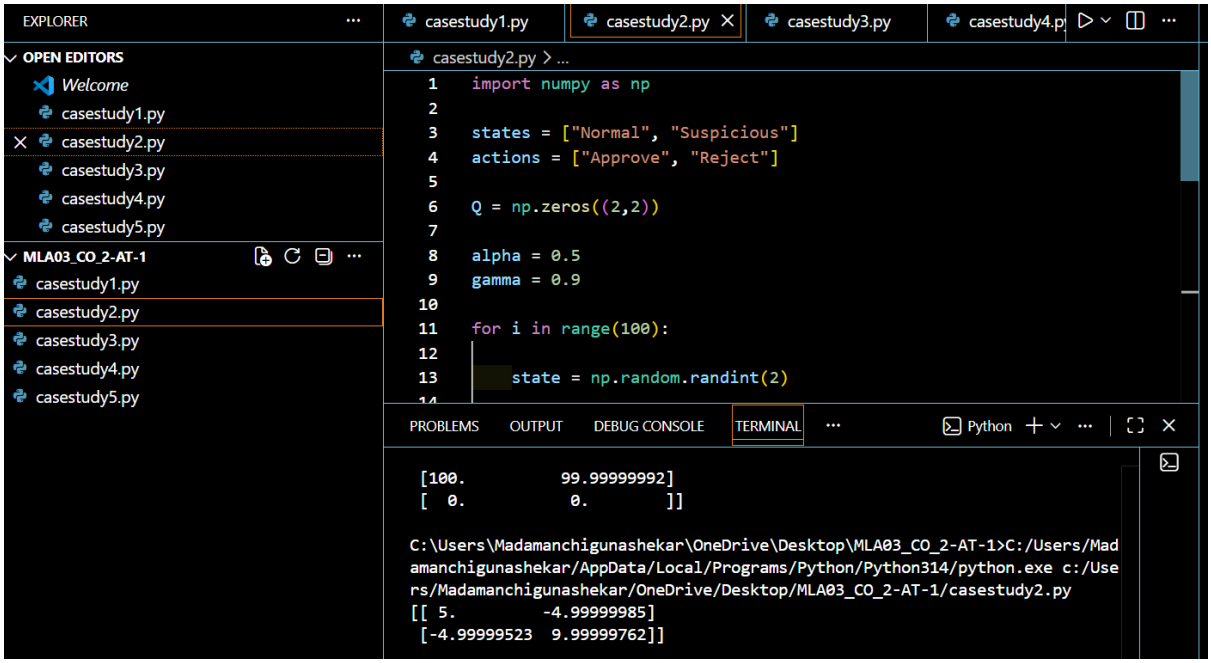
Algorithm

1. Initialize Q-table.
2. Observe transaction state.
3. Select action.
4. Execute action.
5. Receive reward.
6. Update Q-value.
7. Repeat.

Expected Output

- Fraud detection
- Reduced false positives
- Improved security

CODE:-



Applications

- Banking
- Credit cards
- UPI payments
- Digital wallets

Advantages

- Adaptive learning
- Better accuracy
- Handles new fraud types
- Fast decision making

Conclusion

RL enhances fraud detection by continuously learning new fraud patterns, improving financial security over time.

CASE STUDY 3: Reinforcement Learning for Content Recommendation

Title

Personalized Content Recommendation Using Reinforcement Learning

Aim

To recommend personalized content that maximizes user engagement and satisfaction.

Objective

- Recommend relevant content.
- Increase watch time.
- Improve user experience.
- Learn changing user interests.
- Increase platform engagement.

Problem Statement

Traditional recommendation systems struggle to adapt to changing user preferences. Reinforcement Learning continuously improves recommendations using user feedback.

Software Requirements

- Python
- NumPy
- Pandas

Process

1. Collect user profile.
2. Observe viewing history.
3. Recommend content.
4. Collect user feedback.
5. Calculate reward.
6. Update policy.
7. Repeat.

State Space

- User history
- Preferred category
- Watch duration
- Viewing time

Action Space

- Recommend Movie
- Recommend Series
- Recommend Documentary
- Recommend Music

Reward Function

- +50 → Full watch
- +20 → Like
- -20 → Skip
- -50 → Immediate exit

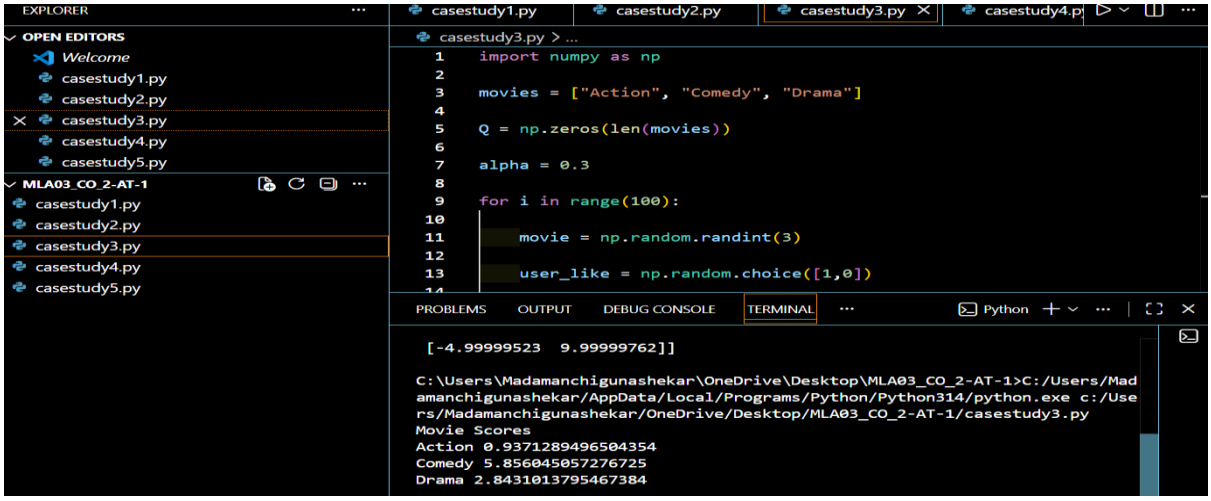
Algorithm

1. Initialize Q-table.
2. Observe user state.
3. Recommend content.
4. Collect feedback.
5. Update reward.
6. Improve policy.
7. Repeat.

Expected Output

- Better recommendations
- Higher watch time
- Increased customer satisfaction

CODE:-



Conclusion

Reinforcement Learning creates intelligent recommendation systems that continuously adapt to user behaviour and improve content relevance.

CASE STUDY 4: Reinforcement Learning for Predictive Maintenance

Title

Predictive Maintenance Using Reinforcement Learning

Aim

To predict equipment failures and schedule maintenance at the optimal time using Reinforcement Learning.

Objective

- Reduce machine failures.
- Minimize maintenance cost.
- Increase equipment life.
- Reduce downtime.
- Improve production efficiency.

Problem Statement

Traditional maintenance follows fixed schedules, which may lead to unnecessary servicing or unexpected breakdowns. RL predicts maintenance needs based on machine conditions.

Software Requirements

- Python
- NumPy
- Matplotlib

Process

1. Read sensor data.
2. Observe machine condition.
3. Select maintenance action.
4. Perform maintenance if needed.
5. Receive reward.
6. Update policy.
7. Repeat.

State Space

- Temperature
- Pressure
- Vibration

- Machine health

Action Space

- Continue operation
- Schedule maintenance
- Stop machine

Reward Function

- +100 → Prevent failure
- +50 → Healthy operation
- -100 → Breakdown
- -20 → Unnecessary maintenance

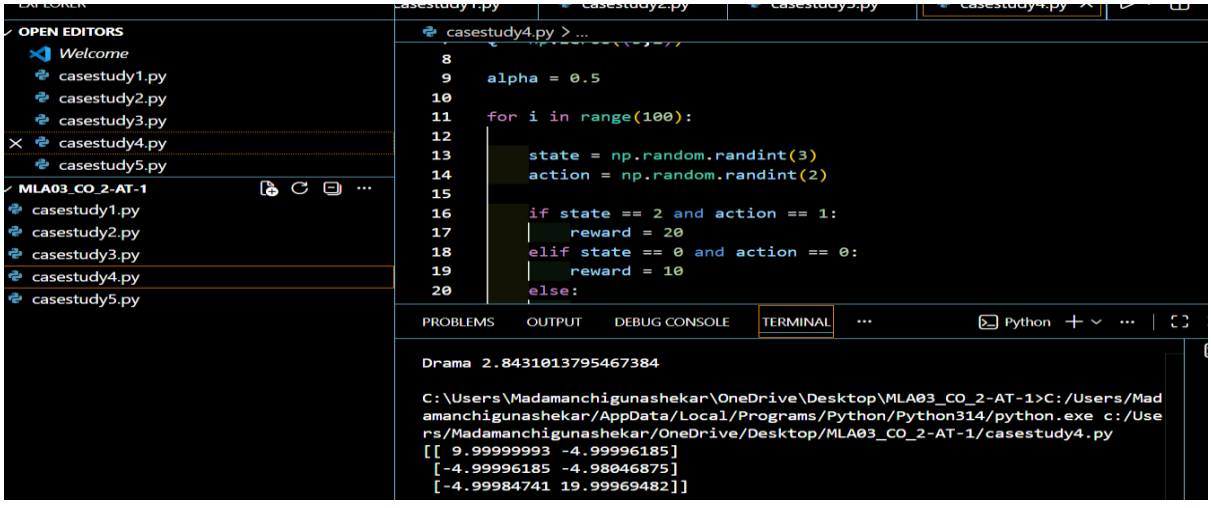
Algorithm

1. Initialize Q-table.
2. Observe machine state.
3. Select action.
4. Execute action.
5. Receive reward.
6. Update Q-value.
7. Repeat.

Expected Output

- Early fault detection
- Reduced downtime
- Improved reliability

CODE:-



Conclusion

RL-based predictive maintenance improves industrial reliability by making maintenance decisions based on real-time machine health.

CASE STUDY 5: Reinforcement Learning for Transportation System Optimization

Title

Transportation System Optimization Using Reinforcement Learning

Aim

To optimize bus scheduling and route allocation using Reinforcement Learning to reduce passenger waiting time and improve transportation efficiency.

Objective

- Reduce passenger waiting time.
- Optimize bus schedules.
- Improve route allocation.
- Reduce traffic congestion.
- Increase vehicle utilization.

Problem Statement

Static bus schedules cannot efficiently respond to changing passenger demand and traffic conditions. Reinforcement Learning continuously learns from real-time data to optimize scheduling and routing decisions.

Software Requirements

- Python
- NumPy
- Matplotlib

Process

1. Collect real-time traffic and passenger data.
2. Observe the transportation system state.
3. Select a scheduling or routing action.
4. Execute the selected action.
5. Receive a reward based on system performance.
6. Update the Q-table.
7. Repeat the learning process continuously.

State Space

- Passenger demand
- Traffic congestion
- Bus location

- Bus occupancy
- Time of day

Action Space

- Assign bus to a route
- Change departure time
- Reallocate buses
- Modify route

Reward Function

- +100 → Reduced passenger waiting time
- +50 → Improved bus utilization
- -50 → Delays
- -100 → Route congestion

Algorithm

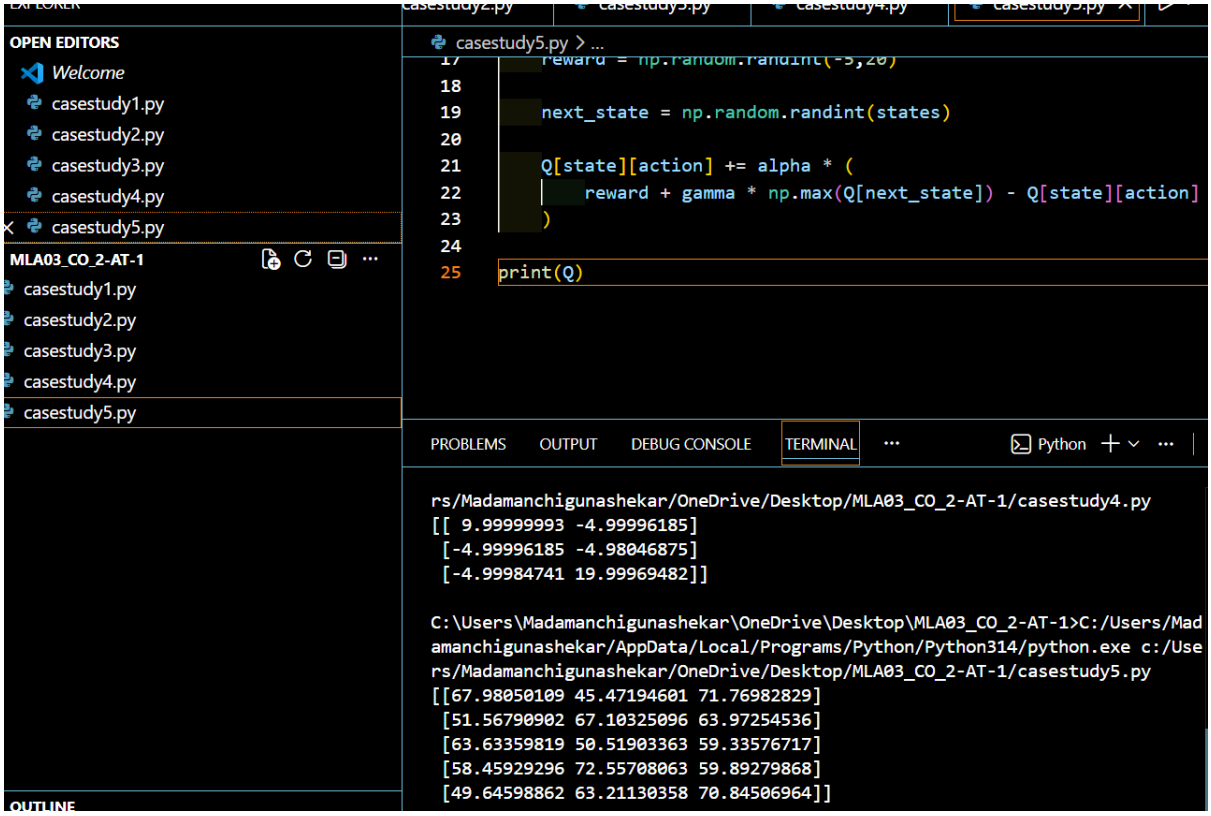
1. Initialize the Q-table.
2. Observe the current transportation state.
3. Select an action using the ε-greedy policy.
4. Execute the action.
5. Receive a reward.
6. Observe the next state.
7. Update the Q-value using the Q-Learning equation.
8. Repeat until an optimal scheduling policy is learned.

Expected Output

- Optimized bus schedules
- Reduced passenger waiting time
- Better route allocation
- Lower operational costs

CODE:-

R



Conclusion

A Reinforcement Learning-based transportation system continuously adapts to traffic conditions and passenger demand, resulting in more efficient scheduling, optimized routes, and improved public transport services.